

GRPO

导论

GRPO 可以概括为以下四步：

1. 同一问题生成多条回答；
2. 对每条回答评分
3. 将奖励转换为组内相对优势
4. 提高优质回答的概率，降低劣质回答的概率

它与 PPO 的区别在于：

$$PPO: \hat{A} = R - V(s)$$

$$GRPO: \hat{A}_i = \frac{r_i - \text{mean}(r_1, \dots, r_G)}{\text{std}(r_1, \dots, r_G) + \varepsilon_{\text{num}}}$$

因此，PPO 使用一个学习得到的价值模型作为参照，而 GRPO 使用同一问题下其他回答的表现作为参照。

“对同一道题生成多条回答，不要求事先知道每条回答的绝对价值，而是根据它在这一组回答中表现得相对好还是相对差来更新模型。”

在传统 PPO 中，通常包含两个模型：

Actor: 语言模型本身，负责生成回答；

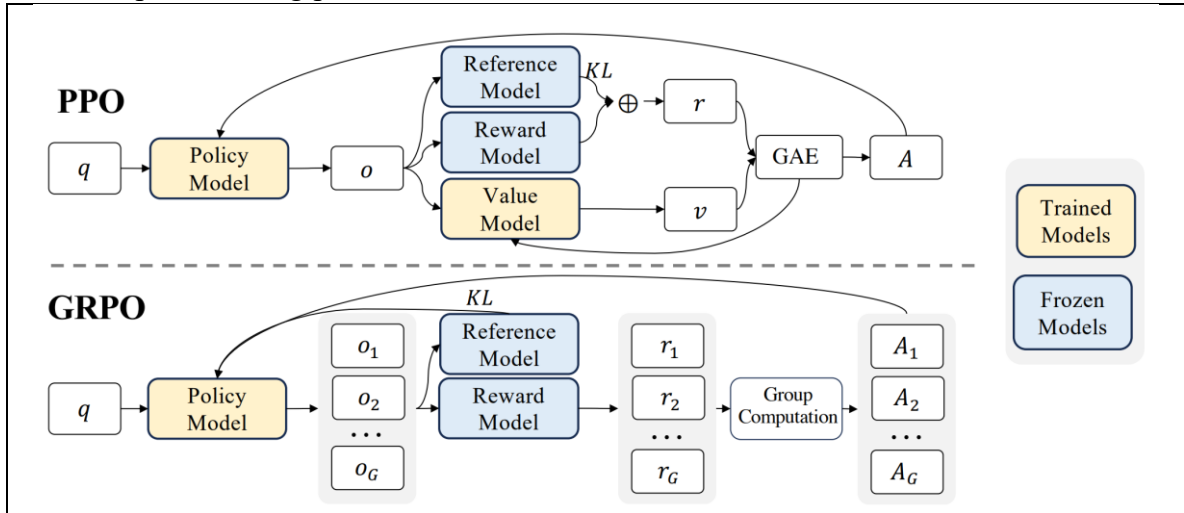
Critic: 价值模型，用于预测当前状态未来能够获得多少奖励。

优势函数一般写为 $A_t = R_t - V(s_t)$ ，其中 R_t 是实际回报； $V(s_t)$ 是 Critic 对状态 s_t 的价值估计； A_t 表示当前动作比预期好多少。

但是对于大语言模型，Critic 通常也是一个规模很大的 Transformer。训练时需要同时保存策略模型、参考模型和价值模型等内容，带来较高的显存和计算成本。

GRPO 不再训练 Critic，而是用同一问题对应的一组回答的平均奖励作为基线。

(<https://arxiv.org/pdf/arXiv:2402.03300>)



给定一个问题 q ，当前旧策略为 $\pi_{\theta_{\text{old}}}$

1. 生成一组回答：对同一个问题采样 G 条回答 $\{o_1, o_2, \dots, o_G\} \sim \pi_{\theta_{\text{old}}}(\cdot | q)$

例如，同一道数学题生成四种不同的解答 o_1, o_2, o_3, o_4 ，这里的 G 称为组大小

2. 计算每条回答的奖励：通过奖励函数或验证器获得 $r_1, r_2, \dots, r_G$ ，对于数学问题，奖励可以根据最终答案是否正确来确定 $r_i = \begin{cases} 1, & \text{答案正确,} \\ 0, & \text{答案错误.} \end{cases}$ ，还可以加入格式奖励，例如要求模型使用特定格式输出最终答案。

3. 计算组内相对优势：首先计算组内平均奖励 $\mu_r = \frac{1}{G} \sum_{j=1}^G r_j$ ，再计算奖励标准

$$\text{差 } \sigma_r = \sqrt{\frac{1}{G} \sum_{j=1}^G (r_j - \mu_r)^2}$$

第 i 条回答的相对优势为 $\hat{A}_i = \frac{r_i - \mu_r}{\sigma_r + \varepsilon_{\text{num}}}$ ，其中 $\hat{A}_i$ 是第 i 条回答的优势； ε_{num}

是防止分母为零的小常数； $\hat{A}_i > 0$ 表示该回答优于组内平均水平； $\hat{A}_i < 0$ 表示该回答劣于组内平均水平。

优势取决于回答在当前组中的**相对位置**。（Group Relative）

[假设同一道题生成四条回答，其奖励为[1,1,0,0]

组内平均奖励为 $\mu_r = \frac{1+1+0+0}{4} = 0.5$ ，标准差为 $\sigma_r = 0.5$ ，因此四条回答的

优势分别为 $\hat{\mathbf{A}} = [1, 1, -1, -1]$ ，含义是两条正确回答的生成概率应该提高；两条错误回答的生成概率应该降低。模型并不需要额外的 Critic 来预测每一步的价值，而是直接通过回答之间的比较获得训练信号。

GRPO 的目标函数

对于第 i 条回答中的第 t 个词元，定义策略概率比 $\rho_{i,t}(\theta) = \frac{\pi_{\theta}(o_{i,t} | q, o_{i,<t})}{\pi_{\theta_{\text{old}}}(o_{i,t} | q, o_{i,<t})}$

其中 π_{θ} 是正在训练的新策略； $\pi_{\theta_{\text{old}}}$ 是采样回答时使用的旧策略； $o_{i,t}$ 是第 i 条回答中的第 t 个词元； $o_{i,<t}$ 表示该词元之前的所有词元。

GRPO 的裁剪目标可以写为：

$$\mathcal{L}_{\text{clip}}(\theta) = \frac{1}{G} \sum_{i=1}^G \frac{1}{|o_i|} \sum_{t=1}^{|o_i|} \min(\rho_{i,t}(\theta) \hat{A}_i, \text{clip}(\rho_{i,t}(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_i)$$

其中 ϵ 是裁剪范围； $|o_i|$ 是第 i 条回答的词元数量；裁剪机制限制模型一次更新不能偏离旧策略太远。

完整目标还会加入参考模型的 KL 散度约束：

$$\mathcal{J}_{\text{GRPO}}(\theta) = \mathcal{L}_{\text{clip}}(\theta) - \beta D_{\text{KL}}(\pi_{\theta} | \pi_{\text{ref}})$$

其中 π_{ref} 是固定的参考模型； D_{KL} 衡量当前模型与参考模型的差异； β 控制 KL 惩罚强度。KL 惩罚用于防止模型为了追求奖励而过度偏离原始语言能力。

[举例：

对于同一个数学问题，旧策略生成两条回答 $G=2$

假设两条回答的奖励分别为 $r_1 = 1$, $r_2 = 0$

也就是第一条回答正确；第二条回答错误。组内平均奖励为 $\mu_r = \frac{1+0}{2} = 0.5$ ；
组内标准差为 $\sigma_r = 0.5$

于是，两条回答的相对优势分别为 $\hat{A}_1 = \frac{1-0.5}{0.5} = 1$ ； $\hat{A}_2 = \frac{0-0.5}{0.5} = -1$ 。表示
第一条回答表现优于组内平均水平，应当提高其生成概率；第二条回答表现劣于组内
平均水平，应当降低其生成概率。

假设每条回答有三个词元，令 $|o_1| = |o_2| = 3$ ，裁剪参数设为 $\epsilon = 0.2$ ，因此允许的概率比范围 $[1 - \epsilon, 1 + \epsilon] = [0.8, 1.2]$ ，对于超出这个范围的概率比，会被裁剪到边界 $\text{clip}(\rho, 0.8, 1.2)$

概率比定义为 $\rho_{i,t}(\theta) = \frac{\pi_{\theta}(o_{i,t} | q, o_{i,<t})}{\pi_{\theta_{\text{old}}}(o_{i,t} | q, o_{i,<t})}$ ，假设具体概率如下：

回答	词元	旧策略概率	新策略概率	概率比
第一条回答	第一个词元	0.20	0.30	1.50
第一条回答	第二个词元	0.40	0.36	0.90
第一条回答	第三个词元	0.30	0.33	1.10
第二条回答	第一个词元	0.50	0.30	0.60
第二条回答	第二个词元	0.20	0.22	1.10
第二条回答	第三个词元	0.40	0.36	0.90

例如：

第一条回答的第一个词元对应 $\rho_{1,1} = \frac{0.30}{0.20} = 1.50$ ；

第一条回答的第三个词元对应 $\rho_{1,3} = \frac{0.33}{0.30} = 1.10$

第二条回答的第一个词元对应 $\rho_{2,1} = \frac{0.30}{0.50} = 0.60$

第二条回答的第三个词元对应 $\rho_{2,3} = \frac{0.36}{0.40} = 0.90$

第一条回答的优势为 $\hat{A}_1 = 1$ ，第一个词元其概率比为 $\rho_{1,1} = 1.50$

未裁剪项为 $\rho_{1,1}\hat{A}_1 = 1.50 \times 1 = 1.50$ ，由于 1.50 超过上界 1.20，裁剪后为 $\text{clip}(1.50, 0.8, 1.2) = 1.20$ ，裁剪项为 $1.20 \times 1 = 1.20$ ，取两者中较小的值 $\min(1.50, 1.20) = 1.20$ ，这表示第一条回答是正确回答，但模型不能在一次更新中将该词元的概率提高得过多。

第二个词元其概率比为 $\rho_{1,2} = 0.90$ ，未裁剪项为 $\rho_{1,2}\hat{A}_1 = 0.90 \times 1 = 0.90$ ，由于 0.90 位于 $[0.8, 1.2]$ 内，因此 $\text{clip}(0.90, 0.8, 1.2) = 0.90$ ，于是 $\min(0.90, 0.90) = 0.90$

第三个词元其概率比为 $\rho_{1,3} = 1.10$ ，未裁剪项为 $\rho_{1,3}\hat{A}_1 = 1.10 \times 1 = 1.10$ ，由于 1.10 位于 $[0.8, 1.2]$ 内，于是 $\min(1.10, 1.10) = 1.10$

第一条回答包含三个词元，因此需要将三个词元的贡献取平均：

$$\frac{1}{|o_1|} \sum_{t=1}^{|o_1|} \min(\dots) = \frac{1.20 + 0.90 + 1.10}{3}$$

计算得到 $\frac{1.20 + 0.90 + 1.10}{3} = \frac{3.20}{3} \approx 1.067$ ，因此，第一条回答对目标函数的平均贡献约为 1.067

同理可得到第二条回答对目标函数的平均贡献约为-0.933

GRPO 裁剪目标为：

$$\mathcal{L}_{\text{clip}}(\theta) = \frac{1}{G} \sum_{i=1}^G \frac{1}{|o_i|} \sum_{t=1}^{|o_i|} \min(\rho_{i,t}(\theta) \hat{A}_i, \text{clip}(\rho_{i,t}(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_i)$$

将两条回答的结果代入 $\mathcal{L}_{\text{clip}}(\theta) = \frac{1}{2}(1.067 - 0.933)$

因此 $\mathcal{L}_{\text{clip}}(\theta) \approx \frac{0.134}{2} = 0.067$

使用未经四舍五入的数值计算：

$$\mathcal{L}_{\text{clip}}(\theta) = \frac{1}{2} \left(\frac{3.20}{3} - \frac{2.80}{3} \right); \quad \mathcal{L}_{\text{clip}}(\theta) = \frac{1}{2} \times \frac{0.40}{3}$$

最终得到 $\mathcal{L}_{\text{clip}}(\theta) = \frac{1}{15} \approx 0.0667$

各个词元的计算结果汇总如下：

回答	优势	概率比	未裁剪项	裁剪项	最终取值
回答一，词元一	1	1.50	1.50	1.20	1.20
回答一，词元二	1	0.90	0.90	0.90	0.90
回答一，词元三	1	1.10	1.10	1.10	1.10
回答二，词元一	-1	0.60	-0.60	-0.80	-0.80
回答二，词元二	-1	1.10	-1.10	-1.10	-1.10
回答二，词元三	-1	0.90	-0.90	-0.90	-0.90

完整目标为 $\mathcal{J}_{\text{GRPO}}(\theta) = \mathcal{L}_{\text{clip}}(\theta) - \beta D_{\text{KL}}(\pi_{\theta} \mathbf{V} | \pi_{\text{ref}})$

假设 $D_{\text{KL}}(\pi_{\theta} | \pi_{\text{ref}}) = 0.03$

KL 惩罚系数为 $\beta = 0.1$

那么 KL 惩罚为 $\beta D_{\text{KL}} = 0.1 \times 0.03 = 0.003$

最终 GRPO 目标为 $\mathcal{J}_{\text{GRPO}}(\theta) = 0.0667 - 0.003$

因此 $\mathcal{J}_{\text{GRPO}}(\theta) \approx 0.0637$ ，训练过程中需要最大化该目标。

[旧策略记为 $\pi_{\theta_{\text{old}}}$ ，其中 θ_{old} 表示更新前语言模型的全部参数。训练开始时，会复制一份当前模型 $\theta \leftarrow \theta_{\text{old}}$ ，此时 $\pi_{\theta} = \pi_{\theta_{\text{old}}}$ ，随后根据 GRPO 目标函数计算梯度，并更新参数 $\theta \leftarrow \theta + \eta \nabla_{\theta} \mathcal{J}_{\text{GRPO}}(\theta)$ ，其中 η 是学习率； $\nabla_{\theta} \mathcal{J}_{\text{GRPO}}(\theta)$ 是目标函数对模型参数的梯度； θ 更新后得到的新模型就是新策略 π_{θ}

在实际代码中通常将最大化目标改写为最小化负目标 $\mathcal{L}_{\text{train}}(\theta) = -\mathcal{J}_{\text{GRPO}}(\theta)$

然后使用 AdamW 等优化器更新 $\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}_{\text{train}}(\theta)$ ，两种写法本质相同。

语言模型不会直接保存“某个词元的概率”。它首先输出每个词元对应的原始分数，称为 **logit**。

假设某个位置的词表中只有三个词元 A, B, C ，旧策略给出的 **logit** 为 $\mathbf{z}_{\text{old}} = [0, 0, 0]$

经过 **softmax** 后，三个词元的概率相同 $\pi_{\theta_{\text{old}}}(A) = \pi_{\theta_{\text{old}}}(B) = \pi_{\theta_{\text{old}}}(C) = \frac{1}{3}$ ，也就是 $\mathbf{p}_{\text{old}} = [0.333, 0.333, 0.333]$ ，假设模型在这个位置实际采样出了词元 A

假设该回答是正确回答，其优势为 $\hat{A} = 1$ 这意味着 GRPO 希望提高词元 A 在当前上下文中的概率。暂时忽略裁剪和 KL 惩罚，在更新开始的位置，目标函数的梯度方向近似为 $\nabla_{\mathbf{z}} \log \pi_{\theta}(A) = [1 - \pi_{\theta}(A), -\pi_{\theta}(B), -\pi_{\theta}(C)]$ ，代入三个概率都是 $1/3$ ：

$\nabla_{\mathbf{z}} \log \pi_{\theta}(A) = \left[\frac{2}{3}, -\frac{1}{3}, -\frac{1}{3}\right]$ ，假设学习率为 $\eta = 0.3$ ，那么使用梯度上升更新 **logit**：

$\mathbf{z}_{\text{new}} = \mathbf{z}_{\text{old}} + \eta \nabla_{\mathbf{z}} \log \pi_{\theta}(A)$ ，代入数值 $\mathbf{z}_{\text{new}} = [0, 0, 0] + 0.3 \left[\frac{2}{3}, -\frac{1}{3}, -\frac{1}{3}\right]$ ，得到 $\mathbf{z}_{\text{new}} = [0.2, -0.1, -0.1]$ 再经过 **softmax**：

$$\pi_{\theta}(A) = \frac{e^{0.2}}{e^{0.2} + e^{-0.1} + e^{-0.1}} \approx 0.403;$$

$$\pi_{\theta}(B) = \frac{e^{-0.1}}{e^{0.2} + e^{-0.1} + e^{-0.1}} \approx 0.299;$$

$$\pi_{\theta}(C) \approx 0.299$$

因此，更新后的新策略概率为 $\mathbf{p}_{\text{new}} \approx [0.403, 0.299, 0.299]$ ，原来被采样词元 A 的概率从 0.333 增加到 0.403 ，对应的策略概率比为 $\rho = \frac{0.403}{0.333} \approx 1.21$

反之，若是错误回答，同理。]]

GRPO 适合数学和代码推理

数学和代码任务通常存在比较明确的验证方法：数学答案可以进行符号或数值匹配；代码可以通过测试用例执行；格式可以通过规则检查；定理证明可以检查最终结论或关键步骤。

因此可以构造相对客观的奖励函数 $r_i = r_i^{\text{accuracy}} + \lambda_{\text{format}} r_i^{\text{format}}$ ，其中 r_i^{accuracy} 是正确性奖励； r_i^{format} 是格式奖励； λ_{format} 是格式奖励权重。

DeepSeekMath 的实验表明，GRPO 可以在不训练额外 Critic 的情况下改善数学推理表现；DeepSeek-R1 随后将基于可验证奖励的强化学习进一步用于更大规模的推理训练。

GRPO 的局限性

1. 所有回答奖励相同时没有有效信号：

如果一组回答全部正确 $r_1 = r_2 = \dots = r_G = 1$ ；

或者全部错误 $r_1 = r_2 = \dots = r_G = 0$

那么 $\sigma_r = 0$ ，归一化后的优势几乎都为零，模型无法区分哪些回答更好。这通常被称为组内零方差问题。

2. 相对优势不等于绝对质量：

假设一组回答都很差，但其中一条稍微好一点，那么这条回答仍可能获得正优势。因此，GRPO 学习的是组内排序，而不是直接估计回答的绝对价值。

3. 组大小影响训练效果：

当 G 较小时，平均奖励和标准差的估计噪声较大；当 G 较大时，估计更稳定，但生成回答的计算成本也会增加。

4. 奖励函数可能被利用

如果奖励规则设计不完善，模型可能学习到能够获得高分但并不真正正确的输出模式，即奖励投机或奖励黑客。

5. 长回答偏差

如果所有词元共享同一个回答级优势，不同长度回答对梯度的贡献方式可能产生偏差，因此实践中需要仔细设计长度归一化和奖励结构。

GRPO 的一次更新过程可以写成：

1. 旧策略生成回答
2. 计算每条回答的奖励
3. 计算组内相对优势
4. 令当前可训练策略初始等于旧策略
5. 计算新旧策略概率比
6. 计算裁剪目标和 KL 惩罚
7. 反向传播计算梯度
8. AdamW 更新模型参数
9. 更新后的模型成为新策略